

INTERVIEW QUESTIONS TO LAND AN AI ENGINEER JOB

Agent Fundamentals & Architecture

Know what makes an agent different from a workflow or chatbot.

Q01 What risks arise if tool descriptions are ambiguous?

A INTERVIEW-READY ANSWER

Ambiguous tool descriptions are one of the most common sources of agent failures in production. The model cannot use a tool correctly if it does not understand when and how to use it.

POINT	WHAT IT MEANS
Wrong tool selection	The agent calls a tool for inputs it was not designed to handle. A general-purpose search tool called for a task that should use a specialized database tool returns poor results and the agent reasons on bad data.
Incorrect argument generation	If the tool description does not clearly explain what each parameter expects, the model generates arguments that are syntactically valid but semantically wrong. A date parameter described only as "date" might receive "next Tuesday" instead of "2024-03-19" causing a tool failure.

Q01 What risks arise if tool descriptions are ambiguous?

A INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Duplicate or conflicting calls	If two tools have overlapping descriptions, the agent may call both unnecessarily, doubling cost and potentially causing conflicts if both take actions.
Over-reliance or under-reliance	A tool described too broadly gets called for everything, including cases where a direct response was appropriate. A tool described too narrowly gets underused even when it is exactly right.
Cascading failures	A wrong tool call produces a wrong result. The agent reasons on that wrong result and takes further wrong actions. By the time the error is visible to the user, the agent may have taken many steps in the wrong direction.
Fix	Treat tool descriptions as carefully as you treat system prompts. Review them with the same rigor. Test tool selection with 50+ diverse queries before deploying. Add explicit "use this when" and "do not use this when" language.

Q02

What metrics indicate that an agent is behaving unreliably?

A

INTERVIEW-READY ANSWER

Reliability metrics for agents require capturing both output quality and behavioral consistency across the multi-step process.

POINT	WHAT IT MEANS
Task completion rate	What percentage of tasks does the agent complete successfully without human intervention? Below 80% is a reliability problem. Track this by task type to identify where failures cluster.
Step count distribution	How many steps does the agent typically use for a given task type? High variance indicates the agent is non-deterministic in ways that suggest unreliable decision-making. A task that usually takes 5 steps but sometimes takes 20 is behaving erratically on the long cases.
Tool error rate	What percentage of tool calls result in errors? High tool error rate can indicate bad argument generation (a reasoning quality problem) or tool reliability issues (an infrastructure problem). Segment by tool to distinguish.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q02

What metrics indicate that an agent is behaving unreliably?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Retry frequency	How often does the agent retry failed steps? A high retry rate suggests the agent is frequently calling tools incorrectly on the first attempt.
Loop detection triggers	How often is the agent caught in a reasoning loop and stopped by the loop detector? Any nonzero rate requires investigation.
Output faithfulness	For tasks with verifiable outputs, what percentage of final answers are factually correct or grounded in the retrieved context? Declining faithfulness indicates reasoning degradation.
User escalation rate	How often do users override, correct, or escalate the agent's actions? Rising escalation rate is a leading indicator of reliability decline.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q03

When is multi-agent collaboration genuinely useful?

A

INTERVIEW-READY ANSWER

Multi-agent systems add complexity. They are only justified when that complexity produces capabilities or efficiency gains that a single agent genuinely cannot match.

POINT	WHAT IT MEANS
Parallelizable tasks	When a task can be broken into independent sub-tasks with no dependencies, running them simultaneously across multiple agents produces a linear speedup. Research tasks with multiple independent queries, data collection across multiple sources, and batch document processing all benefit from parallelization.
Specialized expertise	When different parts of a task require very different capabilities or knowledge, specialized agents outperform a single generalist agent. A pipeline that includes a code-writing agent, a code-reviewing agent, and a documentation agent produces better results than a single agent trying to do all three.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q03 When is multi-agent collaboration genuinely useful?

A INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Scale beyond single context window	When the total information required for a task exceeds a single context window, multiple agents each operating on a subset and reporting summaries to an orchestrator can handle the full scope.
Quality through adversarial collaboration	A generator agent and a critic agent in a debate-style architecture can produce higher-quality outputs than either alone. The critic's adversarial pressure forces the generator to produce more defensible outputs.
When single-agent is sufficient	Sequential tasks without parallelism, tasks where context sharing between steps is critical (splitting context across agents loses coherence), tasks where the coordination overhead exceeds the parallelism benefit, and any situation where you do not have a clear answer to "why can a single agent not do this?"

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q04

When is reflection unnecessary?

A

INTERVIEW-READY ANSWER

Reflection adds cost and latency. For most tasks, a well-designed initial generation with good prompting is sufficient.

POINT	WHAT IT MEANS
Deterministic, rule-based tasks	When the output follows a fixed template and correctness can be verified programmatically (JSON schema validation, regex checks, unit tests for code), there is no need for LLM-based reflection. Just run the validation and regenerate with the error feedback if it fails.
High-confidence outputs	When the model's output confidence is high (determined by the faithfulness of the response to the input, format compliance, and consistency with expected patterns), reflection is unlikely to improve it and only adds cost.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q04 When is reflection unnecessary?

A INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Simple factual lookups	"What is the return policy?" answered by retrieving and paraphrasing the return policy document does not benefit from reflection. The answer is either correct (the policy is correctly retrieved and stated) or incorrect (wrong retrieval, which reflection cannot fix).
Real-time interactions	When users expect sub-second or near-real-time responses, reflection is incompatible with the latency requirements. Speed must take priority.
Low-stakes outputs	Internal logging, draft emails that users will review before sending, preliminary research notes. If the user is going to review and can correct the output, reflection before delivery adds cost without adding net value.

Q05 What defines autonomy in an AI agent, and how do you measure it?

A INTERVIEW-READY ANSWER

Autonomy is the degree to which an agent can accomplish a task without human intervention. It is not binary. It exists on a spectrum.

DIMENSIONS OF AUTONOMY

POINT	WHAT IT MEANS
Decision-making scope	Does the agent decide what actions to take, or does it only execute prescribed steps? An agent with high autonomy chooses its own action sequence. Low autonomy agents follow a human-defined plan.
Error recovery	Can the agent handle unexpected situations without escalating to a human? An autonomous agent detects failures, adapts its approach, and continues. A low-autonomy agent stops and asks for help whenever anything deviates from the expected path.
Goal interpretation	Can the agent interpret a high-level goal into concrete actions? "Research our competitors" is ambiguous. A highly autonomous agent decomposes this into specific searches, decides what to look for, and determines when enough information has been gathered.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q05

What defines autonomy in an AI agent, and how do you measure it?

A

INTERVIEW-READY ANSWER · CONTINUED

HOW TO MEASURE IT

POINT	WHAT IT MEANS
Task completion rate without intervention	Give the agent 100 standardized tasks and measure how many it completes correctly without any human input. Higher percentage = higher autonomy.
Intervention frequency	In production, how often does the agent request human input or fail in a way that requires human recovery? Lower intervention rate = higher autonomy.
Scope of tasks handled	How varied and complex are the tasks the agent can handle independently? An agent that only handles one narrow task category has low autonomy scope even if it handles that category reliably.
Clarification rate	How often does the agent ask clarifying questions vs proceeding based on its own interpretation? Neither extreme is right but the balance reveals autonomy level.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q06

When would a reactive agent outperform a planning-based agent?

A

INTERVIEW-READY ANSWER

REACTIVE AGENTS OUTPERFORM PLANNING AGENTS WHEN



Low-latency systems



Systems requiring responses within tens of milliseconds



Examples: fraud detection, robotics control loops



Deterministic tasks



Known input-output mapping



Example: email classification



High-frequency decision systems



Industrial automation



streaming monitoring

ENVIRONMENTS THAT CHANGE FASTER THAN PLANNING CYCLES

Planning agents are better when tasks require reasoning, sequencing, or exploration.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q07

How do you prevent memory contamination across sessions?

A

INTERVIEW-READY ANSWER

Memory contamination is when one user's or one session's memories inappropriately influence another user's or session's reasoning.

Strict user isolation: Every memory entry is tagged with a user ID. Retrieval queries always include a mandatory user ID filter. No user can ever retrieve another user's memories, regardless of semantic similarity. This is enforced at the data layer, not the application layer.

```
DEF RETRIEVE_MEMORIES(USER_ID: STR, QUERY: STR) -> LIST
```

```
    return memory_store.search(  
        query=query,  
        filters={"user_id": user_id}, # mandatory, cannot be bypassed  
        top_k=5  
    )
```

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q07 How do you prevent memory contamination across sessions?

A INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Session isolation	Short-term memory from one session must not automatically flow into the next session. Session IDs are separate from user IDs. Within a session, the agent has access to full context. Across sessions, only explicitly saved long-term memories are accessible.
Memory write validation	Before writing to long-term memory, validate that the content is appropriate to persist and tagged with the correct user and session identifiers. An automated check that every memory write includes a user ID prevents untagged global memories.
Multi-tenant architecture	For systems serving multiple organizations, maintain separate memory stores per tenant, not just per user. A query by User A at Company X should never access memories from User B at Company Y, even if they asked identical questions.
Audit logging	Log every memory read and write with timestamps, user IDs, and content hashes. This enables investigation of contamination incidents and supports compliance requirements.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q08

How does task decomposition improve agent reliability?

A

INTERVIEW-READY ANSWER

Task decomposition breaks a complex goal into smaller, verifiable sub-tasks. This is the single most important technique for improving agent reliability on complex tasks.

POINT	WHAT IT MEANS
Reduces cognitive load per step	Each decomposed step is simpler than the whole task. A simpler step is easier for the model to reason about correctly and easier for tools to execute reliably.
Creates verification points	Each sub-task has a defined output that can be verified before proceeding to the next. A complex task executed in one shot cannot be verified until the very end. With decomposition, failures are caught early, before downstream work builds on bad intermediate results.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q08

How does task decomposition improve agent reliability?

A INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Enables parallel execution	Independent sub-tasks can be executed simultaneously by separate executor agents. A research task decomposed into five independent searches can run all five in parallel, reducing total time by up to 5x.
Improves error recovery	When a decomposed sub-task fails, only that sub-task needs to be retried or replanned. A single-shot task failure requires restarting everything. Decomposition limits the blast radius of individual failures.
Makes progress visible	Sub-task completion provides natural progress checkpoints. The user can see that 3 of 7 sub-tasks are complete. This is more informative than "the agent is working" for a monolithic task.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q08

How does task decomposition improve agent reliability?

A

INTERVIEW-READY ANSWER · CONTINUED

HOW TO DECOMPOSE EFFECTIVELY

Each sub-task should have clear input requirements and output definition



Sub-tasks should be as independent as possible (minimize dependencies)



Sub-tasks should be sized to fit comfortably within one LLM call plus tool execution



Dependencies between sub-tasks should be explicit and respected in execution order

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q09

How does critique-based refinement improve agent outputs?

A

INTERVIEW-READY ANSWER

Critique-based refinement is a specific reflection pattern where the agent generates an initial output, then generates a structured critique of that output, and finally generates a revised output incorporating the critique.

THE THREE-STEP LOOP

STEP 1 - GENERATE INITIAL OUTPUT



"The refund policy allows returns within 30 days of purchase."

STEP 2 - GENERATE CRITIQUE



"The initial answer is incomplete. It does not specify whether returns



require original packaging, what the refund method is (store credit vs

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q09

How does critique-based refinement improve agent outputs?

A

INTERVIEW-READY ANSWER · CONTINUED



original payment method), or whether sale items are eligible."

STEP 3 - GENERATE REFINED OUTPUT



"The refund policy allows returns within 30 days of purchase.
Returns

REQUIRE ORIGINAL PACKAGING AND RECEIPT. REFUNDS ARE ISSUED TO THE

original payment method within 5 business days. Sale items are final sale."

Why it works: The model in "critique mode" applies different evaluation criteria than in "generation mode." Generation mode optimizes for a plausible, fluent response. Critique mode explicitly looks for errors, omissions, and quality gaps. These different objectives activate different reasoning patterns.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q09

How does critique-based refinement improve agent outputs?

A

INTERVIEW-READY ANSWER · CONTINUED

PRACTICAL IMPROVEMENTS

POINT	WHAT IT MEANS
Completeness	critiques frequently identify missing elements that the initial generation overlooked
Accuracy	critiques catch claims that are unsupported or contradicted by the retrieved context
Format	critiques identify structural issues in the output before the user sees them
Cost management	Run critique-based refinement selectively: only when initial output confidence is low, when the task is high-stakes, or when an automated check flags the initial output.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q10

How is an AI agent architecturally different from a standard LLM-based conversational system?

A

INTERVIEW-READY ANSWER

A traditional LLM chatbot typically performs single-step generation: it receives input and produces text output. Some chatbots may maintain conversation history but still operate in a single inference cycle per response.



An AI agent, however, is designed to operate through multi-step decision loops and interact with external systems.

KEY ARCHITECTURAL DIFFERENCES



Chatbot (Typical LLM interaction)



Input → LLM → Output

USUALLY ONE REASONING STEP PER RESPONSE

May maintain conversation context but rarely performs autonomous task execution

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q10

How is an AI agent architecturally different from a standard LLM-based conversational system?

A

INTERVIEW-READY ANSWER · CONTINUED



Limited or predefined tool usage



AI Agent



Operates in a looped control system



Can plan, take actions, observe results, and iterate



Uses external tools, APIs, databases, or code execution



Pursues goals rather than only responding to prompts

EXAMPLE

POINT

WHAT IT MEANS

Chatbot User

"What is the weather in London?" → LLM answers using knowledge or retrieval.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q10

How is an AI agent architecturally different from a standard LLM-based conversational system?

A INTERVIEW-READY ANSWER · CONTINUED

POINT

WHAT IT MEANS

Agent User

"Book a flight to London next Tuesday."

AGENT WORKFLOW



Understand task



Check calendar



Search flights



Compare options



Ask confirmation

BOOK TICKET

Agents therefore differ primarily in system architecture and control flow, not just the presence of an LLM.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q11

How do you decide whether a task needs an agent or a simple workflow?

A

INTERVIEW-READY ANSWER

This is the most important design question in agent systems. The default should always be the simplest thing that works.

USE A SIMPLE WORKFLOW WHEN



The steps to complete the task are always the same and known in advance

THE TASK HAS NO BRANCHING LOGIC BASED ON INTERMEDIATE RESULTS

Each step is deterministic and the output of each step fully determines the next



Reliability and predictability matter more than flexibility

USE AN AGENT WHEN



The steps to complete the task depend on intermediate results that cannot be known in advance

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q11 How do you decide whether a task needs an agent or a simple workflow?

A INTERVIEW-READY ANSWER · CONTINUED



The task requires dynamic decision-making about which actions to take based on what is discovered

DIFFERENT INPUTS GENUINELY REQUIRE DIFFERENT APPROACHES

The task involves recovering from unexpected failures in ways that cannot be hardcoded

POINT	WHAT IT MEANS
Decision test	Can you draw a flowchart with all possible paths before you receive the actual task input? If yes, use a workflow. If the flowchart requires nodes that say "decide based on the result," you need agent behavior for at least those nodes.
Hybrid approach	Most production systems benefit from a hybrid: a deterministic workflow for the happy path and agent behavior only for exception handling and recovery. This gives you the reliability and predictability of workflows with the flexibility of agents where flexibility is actually needed.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q12

How do you ensure that execution faithfully follows the generated plan?

A

INTERVIEW-READY ANSWER

Plan-execution fidelity is the degree to which the executor's actions actually match what the plan specified. Gaps between plan and execution are a major reliability problem.

STRUCTURED PLAN FORMAT: PLANS SHOULD BE MACHINE-READABLE STRUCTURES, NOT NATURAL LANGUAGE DESCRIPTIONS

```
{
  "steps": [
    {
      "step_id": 1,
      "action": "search_database",
      "parameters": {"query": "customer orders", "filters":
{"status": "pending"}},
      "expected_output": "list of pending orders",
      "on_failure": "escalate"
    }
  ]
}
```



Structured plans leave no ambiguity for the executor about what action to take.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q12 How do you ensure that execution faithfully follows the generated plan?

A INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Step-by-step verification	After each execution step, verify that the action taken matches the planned step. This can be automated: compare the actual tool call made against the planned tool call and flag mismatches.
Plan checkpointing	The executor records each completed step with its actual outcome. The planner reviews the checkpoint and verifies alignment before authorizing the next step.
Executor constraints	The executor's system prompt should constrain it to only execute the current step, not to reinterpret or deviate from the plan: "Execute exactly the step provided. Do not skip steps. Do not modify the plan. If the step cannot be executed, report the failure rather than attempting an alternative."
Deviation logging	Any case where the executor takes an action not in the plan should be logged as a deviation event for review, even if the deviation seemed reasonable.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q13

How does cost scale with reasoning depth?

A

INTERVIEW-READY ANSWER

REASONING DEPTH INCREASES COST DUE TO TWO FACTORS



Additional LLM calls



Growing context window

IF EACH STEP APPENDS TOKENS TO CONTEXT



Step 1 → 1000 tokens



Step 2 → 1500 tokens

STEP 3 → 2000 TOKENS

Each subsequent call processes all previous tokens, making cost growth closer to quadratic in worst cases.

THIS IS WHY PRODUCTION AGENTS OFTEN USE



context pruning



state summarization



scratchpad compression

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q14

How would you design a reflection loop without doubling cost?

A

INTERVIEW-READY ANSWER

Full reflection (generate output + critique + revise) triples the LLM calls for a single task. This is acceptable for high-value tasks but not for routine operations. Smart reflection design keeps the benefit while managing cost.



Conditional reflection: Only trigger reflection when the initial output does not meet a quality threshold:

```
initial_output = generate(prompt)
quality_score = evaluate(initial_output, criteria)
```

IF `QUALITY_SCORE < THRESHOLD`

```
critique = generate_critique(initial_output)
final_output = revise(initial_output, critique)
```

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q14

How would you design a reflection loop without doubling cost?

A

INTERVIEW-READY ANSWER · CONTINUED

ELSE

FINAL_OUTPUT = INITIAL_OUTPUT

If 70% of outputs pass the quality check, reflection is only invoked for 30% of requests, keeping average cost close to 1.3x rather than 3x.

POINT	WHAT IT MEANS
Lightweight critique model	Use a smaller, faster model for critique generation. The critique model only needs to identify problems, not generate high-quality responses. GPT-4o mini or a fine-tuned smaller model for critique can reduce the reflection cost by 80 to 90%.
Focused critique prompts	A general "critique this output" reflection is less efficient than targeted criteria: "Check only: (1) is every factual claim cited? (2) are all required fields present? (3) is the tone appropriate?" Focused critiques are shorter and faster to generate and process.
One-pass revision	After one critique, generate the revision and commit. Do not run another reflection pass on the revision. Accept that one pass of refinement is the cost-effective limit.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q15

How do you prevent coordination conflicts in multi-agent systems?

A

INTERVIEW-READY ANSWER

Coordination conflicts occur when multiple agents attempt to modify the same resource, interpret the same task differently, or produce outputs that contradict each other.

Explicit task ownership: Each task is assigned to exactly one agent at a time. The orchestrator maintains a task registry with current owner and status. An agent must claim a task before working on it, and the claim is exclusive:



Agent A claims task_7 → task_7 locked to Agent A → Agent B cannot also claim task_7

POINT

WHAT IT MEANS

Resource locking

When agents write to shared resources (databases, files, external APIs), implement optimistic or pessimistic locking. The first agent to acquire the lock proceeds; others wait or fail gracefully.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q15

How do you prevent coordination conflicts in multi-agent systems?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Consistent state snapshots	When multiple agents read shared state to make decisions, ensure they read from a consistent snapshot rather than a potentially-in-progress state. Use transactional reads where possible.
Idempotent operations	Design agent actions to be idempotent where possible. If two agents accidentally execute the same step, the result is the same as one agent executing it once. This prevents duplicate side effects from coordination errors.
Conflict detection on write	When an agent writes a result, check for conflicts with previously-written results from other agents. If Agent A reports that Customer X is inactive and Agent B reports Customer X is active, this conflict must be detected and resolved before either result is used.
Explicit dependency graph	When tasks have dependencies, encode them explicitly. Agent B cannot start task_8 until Agent A completes task_5. The orchestrator enforces this order rather than hoping agents will naturally sequence correctly.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Q16

How would you design a tool registry for an agent?

A

INTERVIEW-READY ANSWER

A tool registry is a centralized catalog that defines, stores, and serves tool definitions to agents. It is the infrastructure layer that makes tool management tractable at scale.

CORE COMPONENTS

TOOL DEFINITION SCHEMA: EVERY TOOL IS REGISTERED WITH A STANDARDIZED SCHEMA

```
{
  "name": "search_customer_records",
  "version": "2.1.0",
  "description": "Query the customer database by ID, email, or name...",
  "parameters": {
    "query_type": {"type": "enum", "values": ["id", "email", "name"]},
    "query_value": {"type": "string", "max_length": 200}
  },
  "returns": {"type": "object", "schema": "CustomerRecord"},
  "requires_permission": "customer_data_read",
  "is_idempotent": true,
  "timeout_ms": 2000,
  "rate_limit": "100/minute"
}
```

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q16

How would you design a tool registry for an agent?

A INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Versioning	Tools change. A tool registry maintains multiple versions so agents can be pinned to a specific version. Deprecation warnings allow gradual migration.
Permission system	Each tool requires specific permissions. The agent is granted a set of permissions at initialization based on the user's authorization level. At runtime, the registry only returns tools the agent has permission to use.
Dynamic tool loading	Agents get relevant tools for their task context, not all registered tools. Loading 50 tool descriptions into every prompt is wasteful and confusing. The registry returns only the tools relevant to the current task domain.
Health monitoring	The registry tracks tool availability, error rates, and latency. Unhealthy tools are temporarily removed from the set returned to agents to prevent repeated failures.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q17

Explain the reasoning-act-observe loop and its advantages.

A

INTERVIEW-READY ANSWER

ReAct (Reasoning and Acting) is a prompting and architecture pattern where the agent interleaves reasoning steps with action steps in an explicit, structured loop.

THE LOOP

POINT	WHAT IT MEANS
Thought	I need to find the current price of AAPL stock.
Action	<code>web_search(query="AAPL stock price today")</code>
Observation	AAPL is currently trading at \$189.42 as of 2:30 PM EST.
Thought	I now have the current price. The user also asked about the 52-week high.
Action	<code>web_search(query="AAPL 52-week high 2024")</code>
Observation	AAPL 52-week high is \$199.62.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q17 Explain the reasoning-act-observe loop and its advantages.

A INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Thought	I have both pieces of information needed to answer the question.
Final Answer	AAPL is currently trading at \$189.42. Its 52-week high is \$199.62.

ADVANTAGES

POINT	WHAT IT MEANS
Transparency	Every reasoning step is explicit and logged. You can see exactly why the agent took each action. Debugging is vastly easier than with black-box decision making.
Grounded reasoning	Observations from actual tool calls ground the reasoning in real data. The agent cannot hallucinate a search result because the search result is an observed fact, not a generated one.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q17

Explain the reasoning-act-observe loop and its advantages.

A

INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Error recovery	When an action produces an unexpected result, the thought step processes that observation and can adapt the plan. The loop naturally accommodates surprises.
Step-by-step validation	Each thought-action-observation triplet can be independently validated. Automated monitoring can check whether the observation is consistent with what the action should have returned.
Controllability	Maximum reasoning steps can be enforced at the loop level. Safety checks can be inserted between action and execution. Human approval can be requested at specific steps.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q18

What is self-reflection in agents, and when is it beneficial?

A

INTERVIEW-READY ANSWER

Self-reflection is the agent's ability to evaluate its own outputs, reasoning, or plans and identify potential errors, gaps, or improvements before or after taking action.

How it works: The agent generates an initial response or plan, then runs a second reasoning pass that critiques the first: "Is this plan correct? Are there gaps? Is this answer grounded in the available information? Did I miss anything?"

WHEN IT IS BENEFICIAL

POINT

WHAT IT MEANS

High-stakes
outputs

Before generating a final answer that will be shown to a user or used in a consequential decision, a reflection pass that checks for errors is worth the extra cost. A customer-facing answer that is wrong costs more in trust than the reflection LLM call costs in tokens.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Q18

What is self-reflection in agents, and when is it beneficial?

A

INTERVIEW-READY ANSWER · CONTINUED

POINT	WHAT IT MEANS
Complex multi-step plans	Before executing a 15-step plan, a reflection pass that identifies logical gaps, missing steps, or ordering errors prevents costly execution failures. Catching a plan error before any execution is far cheaper than catching it at step 12.
Ambiguous task interpretation	When the agent is uncertain about how to interpret a request, reflection can surface multiple interpretations and evaluate which is most likely correct before committing to one.
Quality thresholds not met	When an automated quality check (faithfulness score, schema validation) flags an output as below threshold, reflection is a cost-effective way to attempt improvement before escalating to a human.
When it is not worth the cost	Simple factual lookups, deterministic formatting tasks, or cases where the initial output already passes quality checks.

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

This is only a selected preview.

Get the complete Agentic AI Interview Kit for deeper preparation across the full curriculum.

 Full question bank

 Detailed answers

 Follow-up questions

 Real-world scenarios

↓ Check the caption to enroll